

Git and GitHub: Version Control System Intro (Part 1)

SEED AI Bootcamp — Foundations Track

Module: Developer Tooling

Part: 1 of 2 (this part covers local Git workflow, the essential bash you need, and pushing your first project to GitHub)

Why This Module Exists

Every project you build in this bootcamp, from your first Python script to your final AI capstone, needs a safety net and a paper trail. That is what version control gives you.

- A **safety net**: you can break your code, experiment wildly, and always get back to a working version.
- A **paper trail**: you can see exactly what changed, when, and why.
- A **collaboration layer**: teams work on the same codebase without overwriting each other's work.

Git is the tool that tracks changes on your machine. **GitHub** is the website that hosts a copy of your Git project online so others can see it, use it, or contribute to it. Git and GitHub are not the same thing. Git works with no internet connection at all. GitHub is one of many places you can send your Git history to.

By the end of Part 1, you will be able to:

1. Move around your file system and manage files using bash.
 2. Initialize a Git repository and understand the staging area.
 3. Track changes using `git add`, `git commit`, `git status`, and `git log`.
 4. Connect a local repository to GitHub and push your work.
 5. Create and switch between branches.
-

1. The Bash You Actually Need

Git is normally used through the command line, so before touching Git, you need a handful of bash commands. You do not need to master bash, you need to be dangerous enough with it.

Command	What it does	Example
<code>pwd</code>	Prints your current directory	<code>pwd</code>
<code>ls</code>	Lists files in the current directory	<code>ls -la</code> (shows hidden files too)
<code>cd</code>	Changes directory	<code>cd projects/seed-bootcamp</code>
<code>cd ..</code>	Moves up one directory	<code>cd ..</code>
<code>mkdir</code>	Creates a new folder	<code>mkdir my-first-repo</code>

Command	What it does	Example
<code>touch</code>	Creates a new empty file	<code>touch README.md</code>
<code>cat</code>	Prints a file's content	<code>cat README.md</code>
<code>rm</code>	Deletes a file	<code>rm old-notes.txt</code>
<code>clear</code>	Clears your terminal screen	<code>clear</code>

Try this now

```
pwd
mkdir seed-git-practice
cd seed-git-practice
touch README.md
ls -la
```

If you see `README.md` in the output of `ls -la`, your bash setup is working and you are ready for Git.

2. What Git Actually Does (Mental Model First)

Before commands, build the picture in your head. Git tracks your project through three areas:

```
working Directory  --->  Staging Area  --->  Repository (History)
  (your files)           (git add)           (git commit)
```

- **Working directory:** the actual files on your computer, the ones you edit in VS Code.
- **Staging area:** a holding zone where you place the exact changes you want to save next. This is what `git add` does.
- **Repository:** the permanent, timestamped history of your project. This is what `git commit` creates.

Think of it like packing a box to ship. The working directory is your messy room. `git add` is picking specific items off the floor and putting them in the box. `git commit` is sealing the box and writing a label on it that describes exactly what is inside and when it was packed.

3. Initializing a Repository

```
cd seed-git-practice
git init
```

This creates a hidden `.git` folder. That folder is the entire database of your project's history. Never delete it unless you want to lose all version history.

Check it worked:

```
git status
```

You should see something like:

```
On branch main
No commits yet
nothing to commit (create/copy files and use "git add" to track)
```

4. `git status`: Your Most-Used Command

Run `git status` constantly. It tells you three things at any moment:

1. Which branch you are on.
2. Which files have changed but are not staged.
3. Which files are staged and ready to commit.

```
touch app.py
git status
```

Output:

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  app.py
```

"Untracked" means Git sees the file exists but is not yet watching it for changes.

5. `git add`: Staging Your Changes

```
git add app.py
```

This moves `app.py` from "untracked" into the staging area. Run `git status` again and it now shows under "Changes to be committed."

Useful variations:

Command	Meaning
<code>git add app.py</code>	Stage one specific file
<code>git add .</code>	Stage everything in the current directory
<code>git add *.py</code>	Stage all Python files
<code>git add -A</code>	Stage all changes across the whole repo, including deletions

Guidance: early on, prefer staging files individually or in small deliberate groups. `git add .` is convenient but it is easy to accidentally commit files you did not mean to (like `.env` files with secrets, or bulky dataset files). Discipline here now saves you real pain later.

6. `git commit`: Saving a Snapshot

```
git commit -m "Add initial app.py file"
```

The `-m` flag lets you write your commit message inline. The message should describe **what changed and why**, in the present tense, like an instruction: "Add," "Fix," "Update," not "Added" or "Fixing."

Good commit messages:

```
git commit -m "Add data cleaning function for CSV pipeline"
git commit -m "Fix off-by-one error in tokenizer loop"
git commit -m "Update README with setup instructions"
```

Weak commit messages to avoid:

```
git commit -m "stuff"
git commit -m "fix"
git commit -m "asdf"
```

A commit message is a note to your future self and to your teammates. Write it like someone else has to understand your change without reading the code.

7. `git log`: Reading Your Project's History

```
git log
```

Shows every commit: its unique hash (ID), author, date, and message.

```
commit 8f3a9c1e2b7d4f5a6c9d1e2b3f4a5c6d7e8f9a0b
Author: Gita Fonyu <gita@example.com>
Date:   Tue Jul 21 10:15:32 2026 +0100

    Add initial app.py file
```

Useful variations:

Command	Meaning
<code>git log --oneline</code>	Compact, one line per commit
<code>git log --oneline --graph</code>	Shows branch structure visually
<code>git log -n 5</code>	Shows only the last 5 commits
<code>git log --author="Gita"</code>	Filters commits by author

`--oneline` is the one you will use most day-to-day. Get comfortable with it early.

8. Connecting to GitHub

GitHub does not know your local repository exists until you tell it to. The workflow is:

1. Create an empty repository on GitHub.com (do **not** initialize it with a README if you already have a local project, to avoid conflicting histories).
2. Link your local repo to that GitHub repo as a "remote."
3. Push your commits up.

```
git remote add origin https://github.com/your-username/your-repo-name.git
git branch -M main
git push -u origin main
```

Breaking this down:

- `git remote add origin <url>` tells Git that the nickname "origin" points to that GitHub URL. "origin" is just a convention, not a magic word, but almost everyone uses it.
- `git branch -M main` renames your current branch to `main`, the modern standard default branch name.
- `git push -u origin main` uploads your commits to GitHub and sets `origin main` as the default target for future pushes, so afterward you can simply run `git push`.

Checking your remote:

```
git remote -v
```

This shows the fetch and push URLs currently configured.

9. Branches: Working Without Fear

A branch is an independent line of development. `main` is your stable, working version. When you want to try something new, add a feature, or fix a bug, you create a branch so your experiment cannot break `main` until you are ready to merge it back in.

```
git branch feature-login-page
git checkout feature-login-page
```

Or do both in one step:

```
git checkout -b feature-login-page
```

Check which branch you are on and see all branches:

```
git branch
```

The branch with the asterisk `*` is your current branch.

Once your feature branch has commits and you are ready to bring the work back into `main`:

```
git checkout main
git merge feature-login-page
```

Push a branch to GitHub for the first time:

```
git push -u origin feature-login-page
```

Why branch instead of just committing straight to `main`? Imagine you are mid-way through a risky change and it breaks everything. If you were working on `main`, your entire team's stable code is now broken. If you were working on a branch, `main` is untouched, and you can abandon or fix the branch on your own time.

10. The Core Loop, Put Together

This is the cycle you will repeat constantly, for the rest of your career:

```
git status           # see what changed
git add <files>      # stage the changes you want to save
git commit -m "clear message" # save a snapshot
git push             # send it to GitHub
```

Add branching in when working on something isolated:

```
git checkout -b my-new-feature
# ... do work, edit files ...
git add .
git commit -m "Implement my new feature"
git push -u origin my-new-feature
```

Memorize this loop before memorizing anything else in Git. Almost everything else you will learn later (rebasing, cherry-picking, resolving conflicts) is built on top of this loop.

Exercises

Work through these in order. Do not skip the early ones even if they feel too easy, muscle memory matters more than cleverness at this stage. Use a real terminal, not a mental walkthrough.

Section A: Bash Warm-Up

1. Create a folder called `seed-exercise-1`. Move into it. Print your current path to confirm you are inside it.
2. Inside that folder, create three empty files: `notes.md`, `main.py`, `.gitignore`.
3. List all files in the folder, including hidden ones, in long format.
4. Print the contents of `notes.md` to your terminal (it will be empty, that is fine).

5. Delete `main.py`, then confirm it is gone using `ls`.

Section B: First Repository

6. Initialize a Git repository inside `seed-exercise-1`.
7. Run `git status` before creating any files and write down, in your own words, what it tells you.
8. Create a file called `hello.py` containing a single line: `print("Hello, SEED")`.
9. Run `git status` again. Explain the difference in output compared to exercise 7.
10. Stage `hello.py` and run `git status` a third time. What section does the file now appear under?

Section C: Committing

11. Commit `hello.py` with a clear, descriptive commit message.
12. Run `git log` and identify the commit hash, author, and date of your commit.
13. Create a second file, `utils.py`, with any content you like. Stage and commit it separately from `hello.py`.
14. Run `git log --oneline`. How many commits do you now have?
15. Edit `hello.py` so it prints a different message. Run `git status`. Why does Git see it as "modified" and not "untracked"?
16. Stage and commit the change from exercise 15 with a message describing exactly what changed.

Section D: Reading History

17. Run `git log --oneline --graph`. Describe what you see.
18. Run `git log -n 2`. What does the `-n` flag control?
19. Deliberately write one "bad" practice commit message (e.g. "stuff") for a trivial change, then immediately write down why this message would confuse a teammate six months from now.

Section E: Pushing to GitHub

20. Create a new, empty repository on GitHub named `seed-exercise-1` (do not add a README).
21. Connect your local repository to this GitHub repository using `git remote add`.
22. Confirm the remote was added correctly using `git remote -v`.
23. Push your `main` branch to GitHub for the first time, setting the upstream tracking branch.
24. Refresh the GitHub page in your browser. Confirm your commits and files appear there.
25. Make one more small change locally, commit it, and push again, this time without using the `-u` flag. Explain why it still worked.

Section F: Branching

26. Create a new branch called `feature-greeting` without switching to it yet. Confirm it exists using `git branch`.
27. Switch to `feature-greeting`.

28. While on this branch, edit `hello.py` to greet a specific SEED intern by name. Stage and commit the change.
29. Switch back to `main`. Open `hello.py`. Confirm your greeting change is **not** present on `main`. Explain why.
30. Merge `feature-greeting` into `main`. Open `hello.py` again and confirm the change now appears.
31. Push your updated `main` branch to GitHub.
32. Push `feature-greeting` to GitHub as well, so both branches are visible online.

Section G: Challenge Round

33. Starting from a fresh folder, complete the entire loop from scratch, unaided: init a repo, create at least three files across at least two commits, connect to a new GitHub repo, push, create a branch, make a change on the branch, merge it, and push again. Time yourself. Try to do it in under ten minutes.
34. Explain, in your own words and without looking anything up, the difference between the working directory, the staging area, and the repository.
35. Explain, in your own words, why teams use branches instead of everyone committing directly to `main`.
36. Write three commit messages: one weak, one good, and one excellent, for the same hypothetical change (fixing a bug where a login form crashes on empty input). Explain what separates each tier.

Coming up in Part 2: resolving merge conflicts, `git pull` vs `git fetch`, `.gitignore` in depth, pull requests and code review on GitHub, and forking workflows for open-source contribution.